VoltCast

Household Electricity Consumption Prediction System

Project explanation for the presentation and viva

MLOps-1 (25CSH-320) Mini Project, Chandigarh University

Live deployment	https://voltcast-forecast.vercel.app
GitHub repository	https://github.com/Kunal039/voltcast
Team	Kunal Mittal (24BCS12354), Ansh Kumar (24BCS12097), Vishal Kumar Shrivastava (24BCS12629)

Result	Value
Forecasting model	Random forest (syllabus algorithm)
Forecast error (test MAE)	0.295 kW, 22% lower than the baseline
R^2 (test)	0.620
Accuracy (1 - WAPE)	67.9%
Peak-hour alert model	Random forest classifier
Peak-hour alert accuracy / ROC-AUC	85.7% / 0.910

1. One-line summary (start with this)

> "VoltCast is an MLOps project that predicts how much electricity a household will use over the next 1 to 48 hours, and warns about peak-usage hours. It is not just a model in a notebook: it is a complete, automated system that is versioned, tested, deployed on the cloud and monitored."

Live: https://voltcast-forecast.vercel.app
Code: https://github.com/Kunal039/voltcast

2. The problem (why it matters)

- Households get a monthly bill but no idea how much electricity they will use tomorrow, or when the peaks will be.
- Electricity companies need short-term, household-level forecasts to manage peak demand; India is installing about 250 million smart meters, so this data is becoming common.
- A model built once and left alone becomes inaccurate over time (for example when seasons change). That is why **MLOps** is needed: automation, testing, monitoring and retraining.

3. The data

- **UCI Individual Household Electric Power Consumption** (public): 2,075,259 one-minute readings from one house near Paris, Dec 2006 - Nov 2010.
- Columns: active power (kW), reactive power, voltage, current, and 3 sub-meters (kitchen, laundry, water heater / AC).
- 1.252% of readings were missing: short gaps are interpolated, longer gaps filled with the same hour of the previous week.
- Outliers were checked with the IQR rule and kept, because they are real peak usage.
- Minute readings were converted to **34,589 hourly records**.
- The dataset is in the repository (data/household_power_consumption.csv.gz).

4. Feature engineering

31 input features, all built only from the **past**, so the model never sees the hour it predicts (an automated test checks this):

- Lag features: usage 1, 2, 3, 6, 12, 24, 48 and 168 hours ago (168 h = same hour last week).
- Rolling averages and standard deviations over 3, 6, 24 and 168 hours.
- Calendar: hour, weekday, month, weekend, holiday (with sin/cos encodings).
- Previous hour's readings: voltage, current, reactive power and the three sub-meters.

5. The machine learning (syllabus algorithms only)

Split by time, not randomly: train 2006-2009, validate Jan-May 2010, test Jun-Nov 2010. A random split would leak future data into training and give fake high scores.

Algorithms compared (all from the syllabus): linear regression, ridge regression, decision tree, KNN, SVM and random forest, plus two simple baselines ("same as last hour" and "same hour yesterday").

Model	Train MAE	Validation MAE	Validation R^2
Persistence baseline	0.449	0.441	0.364
Seasonal naive (24 h)	0.663	0.636	-0.184
Linear regression	0.409	0.398	0.559
Ridge regression	0.409	0.398	0.559
Decision tree	0.000	0.506	0.142
K-nearest neighbours	0.347	0.395	0.544
SVM (SVR)	0.296	0.355	0.598
Random forest (deployed)	0.220	0.345	0.632

Hyper-parameter tuning: GridSearchCV with time-series cross-validation (Experiment 4).

Key teaching point - over-fitting: the default decision tree has **0.000 error on training data** but the worst validation error (0.506 kW): it memorised the data. Tuning it (max_depth=10, min_samples_leaf=100) cuts the error to 0.380 kW.

Winner: Random forest (lowest validation error). Results on the unseen test period:

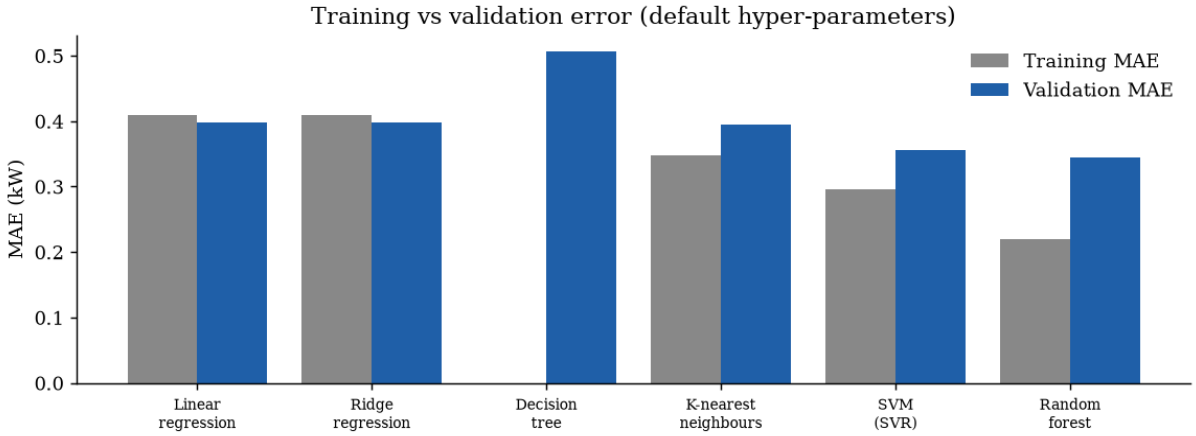
Metric	Value
MAE (average error)	0.295 kW
MSE / RMSE	0.188 / 0.434 kW
R^2	0.620
Accuracy (1 - WAPE)	67.9%
Forecasts within ±0.5 kW	80.8%
Improvement over baseline	21.9% lower error

Second model - peak-hour alert (classification, Experiment 3): predicts whether the next hour is a peak hour (above 1.59 kW, the busiest 25% of hours). Three classifiers were compared:

Classifier (validation)	Train F1	Accuracy	F1	ROC-AUC
Decision tree (default)	1.000	0.770	0.599	0.717
Decision tree (tuned)	0.735	0.790	0.696	0.879
Random forest classifier (selected)	0.875	0.832	0.745	0.910

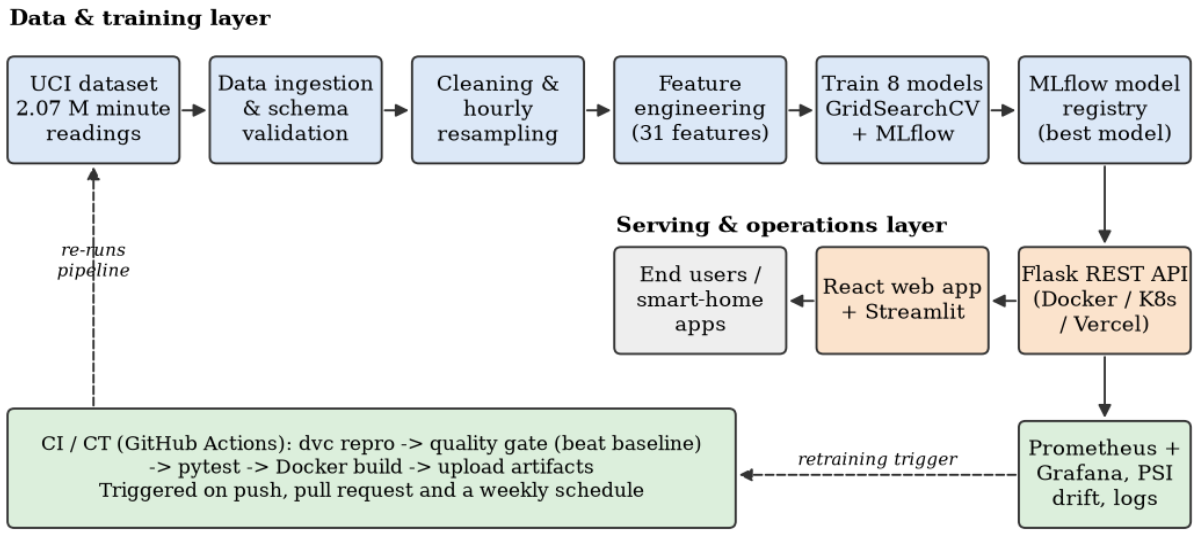
Selected classifier on the test set	Value
Accuracy	85.7%
Precision	56.2%
Recall (share of real peaks caught)	71.8%
F1-score	0.631
ROC-AUC	0.910
Confusion matrix	524 peaks caught, 206 missed, 408 false alarms

If asked "why is accuracy only about 68%?" - "We predict one single house hour by hour; people switching on an oven or heater at random cannot be predicted from past readings. We also used a strict time-based split, which gives honest, lower scores than shuffled splits. About 81% of forecasts are within half a kilowatt, which is the practical measure."



Training vs validation error: the default decision tree's zero training error is over-fitting.

6. The MLOps pipeline (the core of the course)



Step	Tool	What it does	Syllabus
Version control	Git + GitHub	All code tracked; commits throughout the project	Exp 5
Data and pipeline versioning	DVC	One command (dvc repro) reruns everything; dvc.lock records exact versions	Exp 9
Experiment tracking	MLflow	Logs every model's parameters and scores; model registry	Model versioning
API	Flask REST API	/api/forecast, /api/predict, /api/health, /api/metrics; rejects bad input (HTTP 422)	Exp 7
Containerisation	Docker + Compose	5 services: API + web app, dashboard, MLflow, Prometheus, Grafana	Exp 6
Orchestration	Kubernetes	2 replicas, health checks, autoscaling (manifests)	Project-based learning

Step	Tool	What it does	Syllabus
CI/CD	GitHub Actions + Vercel	Every push: rerun pipeline, quality gates, 11 tests, Docker build, auto-deploy	Exp 8
Cloud	Vercel (serverless on AWS)	Public web app and API at https://voltcast-forecast.vercel.app	Unit 3: cloud deployment
Monitoring	Prometheus + Grafana	Live dashboard of requests, speed, predictions, errors	Exp 10
Drift detection	PSI	Flags when new data differs from training data; recommends retraining	Monitoring

Strong point to mention: the drift monitor actually caught real drift. The model trained mostly on winters while the test period was summer; water-heater/AC usage changed a lot (PSI 0.73, above the 0.2 threshold), so the system flagged "retraining recommended" on its own.

7. Ethics (Unit 3)

- **Fairness and bias:** error was checked across groups of hours; it is worse in evenings and high-use hours, and the model is trained on one French house, so it must be retrained before use elsewhere.
- **Privacy:** meter data can reveal when people are home; only a public anonymised dataset is used and the prediction log does not store user readings.
- **Security:** all API input is validated, request size is limited, and rejected requests are counted in monitoring.
- **Transparency:** model card (MODEL_CARD.md), feature importance, and results anyone can reproduce with dvc repro.

8. Live demonstration (about 3 minutes)

1. Open https://voltcast-forecast.vercel.app, pick a date, click **Run forecast**: show the forecast against the actual line and the red peak-hour markers.
2. Scroll to **Model performance**: the comparison table and the over-fitting chart.
3. Show **Peak-hour alerts**: confusion matrix and ROC curve.
4. Open the repository https://github.com/Kunal039/voltcast, then the **Actions** tab: green CI/CD runs.
5. Optional (Docker running): Grafana at localhost:3000 for the monitoring dashboard.

9. Team roles

Member	Role and contribution
Kunal Mittal (24BCS12354)	Data & ML lead: data pipeline, feature engineering, models and tuning, peak-hour classifier, evaluation
Ansh Kumar (24BCS12097)	Deployment lead: Flask API, React web app, Streamlit, Docker, Vercel and Kubernetes
Vishal Kumar Shrivastava (24BCS12629)	MLOps & QA lead: CI/CD, DVC, MLflow, Prometheus and Grafana, testing, report

10. Likely viva questions

Question	Answer
Why random forest?	Lowest validation error of the syllabus algorithms. It averages many decision trees trained on different samples, which fixes the single tree's over-fitting.
Why a time-based split?	A random split lets the model learn from the future and gives fake high scores. Real forecasting always predicts forward in time.
What does DVC do that Git does not?	Versions large data and pipeline outputs by content hash, and re-runs only the stages whose inputs changed.
What happens on git push?	GitHub Actions re-runs the pipeline and tests; Vercel builds and redeploys the live site automatically.
How do you know when to retrain?	If PSI > 0.2 or the error gets 20% worse, the monitor flags it; the pipeline also retrains every week.
Why GridSearchCV with TimeSeriesSplit?	It tries every hyper-parameter combination while every fold trains on the past and validates on the following period.
What is over-fitting here?	The default decision tree: zero training error, worst validation error. Limiting depth and leaf size fixes it.
Precision vs recall for the alert?	Recall 72% means most real peaks are caught; precision 56% means some alerts are false alarms - the safer trade-off for warnings.